

TESTS PASSED PROOF AND COVERAGE REPORT

1) Tests passed when running *mvn clean test*

```
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 10.85 s -- in com.WebSocketPipelineTest
[INFO] Running com.AlertFactoriesTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.015 s -- in com.AlertFactoriesTest
[INFO] Running com.PatientRecordsRetrievalTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.002 s -- in com.PatientRecordsRetrievalTest
[INFO] Running com.SingletonTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.004 s -- in com.SingletonTest
[INFO] Running com.OxygenSaturationStrategyTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.005 s -- in com.OxygenSaturationStrategyTest
[INFO] Running com.AlertDecoratorsTest
[INFO] Tests run: 14, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.012 s -- in com.AlertDecoratorsTest
[INFO] Running com.DataStorageTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0 s -- in com.DataStorageTest
[INFO] Results:
[INFO] Tests run: 44, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] Total time: 15.569 s
[INFO] Finished at: 2026-05-09T20:20:26+02:00
[INFO]
```

2) Coverage report generated

cardio_generator

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods	Missed Classes
com.cardio_generator.generators		0%		0%	24 24	101 101	16 16	5 5
com.cardio_generator		5%		7%	24 27	92 99	10 12	0 1
com.alerts		76%		71%	26 81	50 182	8 34	1 14
com.cardio_generator.outputs		31%		66%	11 20	44 68	10 17	3 5
com.data_management		72%		61%	20 56	44 152	6 29	0 5
com		0%		0%	4 4	5 5	2 2	1 1
Total	1,486 of 2,766	46%	96 of 202	52%	109 212	336 607	52 110	10 31

Report shows that the tests concerning packages *com.alerts* and *com.data_management* containing new classes from parts 3 and 4 were relatively comprehensive. Package *com.cardio_generator.generators* was not tested at all as the outputs of the methods depend on randomness and thus, it is difficult to assert a deterministic correct outcome. Maybe an output range could be checked to verify the behaviour to some extent. Nonetheless, the class would have to be checked multiple times to determine with some degree of certainty that it would not produce an incorrect output as it is non-deterministic. Output strategies (in package *com.cardio_generator.outputs*) and *HealthDataSimulator* were similarly untested as they depend on network connections, file I/O, and scheduled threading which are not suitable for unit testing without mocking frameworks.